

Dorea Fernando - Emanuel

grupa 243

Temă Seminar Algoritmi Aproximativi

Knapsack

ex 1

a) Pseudocod:

$D_p[j]$ = suma maximă pe care o pot obține fără a depăși valoarea totală j

$D_p[j] = 0, \quad \forall j = \overline{0, K}$

for $i = \overline{1, n}$

for $j = \overline{K, s_i}$

$D_p[j] = \max (D_p[j], s_i + D_p[j - s_i])$

soluție : $D_p[K]$

Complexitate

Timpu : $O(n \cdot K)$, Spatiu : $O(K)$

Corectitudine

Algoritmul construiește soluția optimă pas cu pas, stocând în $D_p[j]$ suma maximă ce poate fi obținută pentru valoarea j . La fiecare pas algoritmul alege maximul dintre suma maximă obținută deja anterior și adunarea elementului s_i la cea mai bună sumă calculată $j - s_i$. Parcurgerea descrescătoare de la K la s_i asigură că starea curentă folosește rezultate calculate înainte să adăugăm elementul s_i , evitând să îl adăugăm de mai multe ori.

b)

Pseudocod

Sum = suma curentă

s_{max} = elementul maxim din S

Sum = s_{max} = 0

for $i = \overline{1, n}$

if $s_i > s_{max}$

$s_{max} = s_i$

if $Sum + s_i \leq K$

Sum = Sum + s_i

return : $\max(Sum, s_{max})$

Complexitate

Temp : $O(n)$ Spatiu : $O(1)$

Justificare

Fie OPT soluția optimă a problemei

Conform demonstrației de la curs, știm că OPT este mai mică sau egală decât suma calculată de algoritmul (Sum) plus primul element din sir respins sau depășea capacitatea K

$$OPT \leq Sum + s_k, \quad s_k \in \{s_1, \dots, s_n\}$$

$$s_k \in S \Rightarrow s_k \leq s_{max}$$

$$Alg = \max(Sum, s_{max}) \Rightarrow$$

$$Sum \leq Alg$$

$$s_{max} \leq Alg \quad (+)$$

$$Sum + s_{max} \leq 2 \cdot Alg$$

$$\Leftrightarrow 2 \cdot Alg \geq OPT \Leftrightarrow Alg \geq \frac{1}{2} OPT$$

□

Load Balance

ex 1

$$m = 2$$

$$\text{time total} = 120 + 80 = 200$$

$$\text{ALG} = \text{mașina cu încărcătură maximă} = \max(120, 80) = 120$$

$$\text{ALG} \leq 1.1 \cdot \text{OPT} \text{ posibil?}$$

#

$$a) \quad t_j \leq 100$$

Fie $m = 3$ cu următoarele activități: $\{80, 60, 60\}$

Valoarea optimă: - mașina 1: 80

- mașina 2: 60 60

$$\Rightarrow \text{OPT} = 120$$

$$\text{ALG} \leq 1.1 \cdot \text{OPT}$$

$$120 \leq 1.1 \cdot 120$$

$$1 \leq 1.1 \Rightarrow \text{Adev} \Rightarrow \text{DA, este posibil}$$

$$b) \quad t_j \leq 10$$

+

Conform Teoremei 1 (slide 37) avem că

$$\text{ALG}_{\text{Greedy}} \leq \frac{1}{m} \left(\sum_{j=1}^n t_j \right) + t_2 \left(1 - \frac{1}{m} \right)$$

$$\text{ALG}_G \leq \frac{1}{2} \cdot 200 + 10 \cdot \left(1 - \frac{1}{2} \right)$$

worst case

$$\text{ALG}_G \leq 100 + 10 \cdot \frac{1}{2} = 105 \Rightarrow \text{OPT} \leq 105$$

$$\text{ALG} \leq 1.1 \cdot \text{OPT} \leq 1.1 \cdot 105 = 115.5$$

$$\text{ALG} = 120 < 115.5 \text{ contradicție}$$

$\Rightarrow$ NU, nu este posibil $\square$

ex 2

$$ALG_1(J) \leq 2 \cdot OPT(J), \quad \forall J \text{ input}$$

$$ALG_2(J) \leq 4 \cdot OPT(J), \quad \forall J \text{ input}$$

+

$$a) \quad \exists J \text{ a.i.} \quad ALG_2(J) \geq 2 \cdot ALG_1(J) \\ \#$$

Deoarece ALG_2 este 4 - aproximativ (tight - bound), $\exists$ un input J worst case a.i. $ALG_2(J) = 4 \cdot OPT(J) \quad (1)$

$$ALG_1(J) \leq 2 \cdot OPT(J), \quad \forall J \text{ input}$$

$$\Rightarrow ALG_1(J) \leq 2 \cdot OPT(J) \quad 1 \cdot 2$$

$$2 \cdot ALG_1(J) \leq 4 \cdot OPT(J) \stackrel{(*)}{=} ALG_2(J)$$

$$\Rightarrow 2 \cdot ALG_1(J) \leq ALG_2(J) \quad \Rightarrow \text{Afirmația e } \underline{\text{adeverată}}$$

$$b) \quad \nexists J \text{ a.i.} \quad ALG_1(J) \geq 2 \cdot ALG_2(J) \\ \#$$

$$\nexists J \text{ a.i.} \quad ALG_1(J) \geq 2 \cdot ALG_2(J)$$

$$\Leftrightarrow ALG_1(J) < 2 \cdot ALG_2(J), \quad \forall J \\ \#$$

$$\text{analog} \quad \exists J \text{ a.i.} \quad ALG_1(J) = 2 \cdot OPT(J)$$

Totuși, având în vedere că ALG_2 este tight-bound, nu este exclus ca pe inputul J , outputul să fie cel optim, deci e posibil

$$ALG_2(J) = OPT(J)$$

$$ALG_1(J) < 2 \cdot ALG_2(J), \quad \forall J$$

$$2 \cdot OPT(J) < 2 \cdot OPT(J) \quad \text{contradicție}$$

$\Rightarrow$ afirmație falsă

□

Travelling Salesman Problem

ex 1

a) TSP NP-Hard

#

Fie $G = (V, E)$ un graf oarecare cu n noduri

Construim graful G' unde
$$\begin{cases} w(e) = 1, & \text{dacă } e \in E(G) \\ w(e) = 2, & \text{dacă } e \notin E(G) \end{cases}$$

Lemma 1 : Dacă G are ciclu hamiltonian $\Rightarrow$ soluția optimă în

G' va folosi doar muchii din G , adică cu cost = 1

Deci $OPT = n$

Lemma 2 : Dacă G nu are ciclu hamiltonian $\Rightarrow$ orice traseu în G'

trebuie să folosească măcar o muchie de cost 2

Deci $OPT \geq n+1$

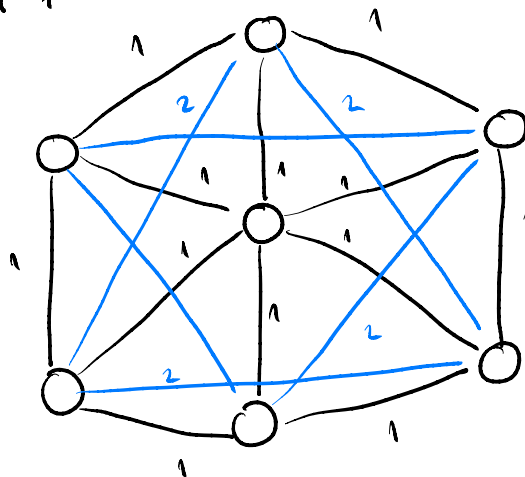
Dacă am rezolva această variantă de TSP în timp polinomial, am putea decide HCP verificând dacă output-ul este n

sau $\geq n+1$. Știm că HCP este NPC, deci problema noastră este

NP-hard.

c) Construim un graf G cu un nod în centru iar restul nodurilor într-un inel exterior.

Muchii din centru către restul nodurilor au cost 1 iar muchiile inelului au tot cost 1



$$OPT = n$$

Când calculăm MST, algoritmul poate alege steaua formată de nodul central și restul nodurilor. Când folosește parcurgerea DFS, poate vizita "frunzele" stelei într-o ordine care sare peste nodurile vecine de pe inel. La conectare, muchiile adăugate nu vor fi cele de pe inel (cost 1), ci rezultatele de cost 2.

$$ALG = 2 \cdot 1 + (n-2) \cdot 2 = 2n - 2$$

$$\frac{ALG}{OPT} = \frac{2n-2}{n} \approx 2 \Rightarrow \text{nu este } \frac{3}{2} \text{ aprox}$$



Algoritmi Aproximativi

(1p oficiu)

Knapsack

(2p)

1. Fie S un șir de numere naturale $\{s_1, s_2, \dots, s_n\}$ și K un număr natural, cu $K \geq s_i$ pentru orice i între 1 și n . Vrem să determinăm suma **maximă**, dar care să fie $\leq K$, ce poate fi formată din elementele din S (numerele pot fi luate cel mult o singură dată).

- a) Scrieți un algoritm pseudo-polinomial care găsește suma optimă. Indicați complexitatea de timp/spațiu a algoritmului propus de voi și justificați de ce acesta este corect (de ce soluția găsită este optimă). (1p)
- b) Scrieți un algoritm aproximativ care calculează o sumă cel puțin pe jumătate de mare ca cea optimă, dar rulează în timp $O(n)$ și complexitate spațiu $O(1)$. (1p)

Load Balance

(3p)

Puteți rezolva, la alegere, cel mult 2 probleme dintre cele 3.

1. Fie o iterație a problemei *Load Balancing* (cursul 2, slide-ul 16) pentru 2 mașini. La seminarul de algoritmi aproximativi unul dintre studenți propune un algoritm de rezolvare și susține că acesta este 1.1 aproximativ. El rulează algoritmul pe un set de n activități și obține o încărcătură de 80 pe una dintre mașini, respectiv 120 pe cealaltă. Este posibil ca factorul lui de aproximare să fie corect...

- a) ...ținând cont că rezultatul obținut anterior a fost făcut pe un set de activități, fiecare cu timpul de lucru cel mult 100? (0,5p)
- b) ...ținând cont că rezultatul obținut anterior a fost făcut pe un set de activități, fiecare cu timpul de lucru cel mult 10? (0,5p)

2. Fie ALG_1 și ALG_2 doi algoritmi de rezolvare pentru aceeași problemă de **minimizare**. ALG_1 este un algoritm 2-aproximativ, respectiv ALG_2 este un algoritm 4-aproximativ (presupunem că ambii factori de aproximare sunt *tight bound*). Stabiliți valoarea de adevăr a următoarelor propoziții, dând și o scurtă justificare:

a) Există cu siguranță un input I pentru care

$$\text{ALG}_2(I) \geq 2 \cdot \text{ALG}_1(I)$$

(0,5p)

b) Nu există niciun input I pentru care

$$\text{ALG}_1(I) \geq 2 \cdot \text{ALG}_2(I)$$

(0,5p)

3. Fie algoritmul *Ordered-Scheduling Algorithm* (cursul 8, slide-ul 42), care implică algoritmul descris anterior (slide-ul 19) la care adăugăm o preprocesare cu care sortăm descrescător activitățile după timpul de desfășurare. Th. 2 afirmă că acest algoritm este $\frac{3}{2}$ -aproximativ. Arătați că acest factor de aproximare poate fi îmbunătățit la $\frac{3}{2} - \frac{1}{2m}$ (unde m este numărul de calculatoare pe care se pot executa activități).

(2p)

Travelling Salesman Problem

(2p)

Rezolvați, la alegere, una dintre cele trei probleme.

1. Considerăm o variantă a TSP, în care toate muchiile din graf au ponderea 1 sau 2.

a) Arătați că problema rămâne NP-hard pentru aceste instanțe. (1p)

b) Arătați că aceste ponderi satisfac în continuare inegalitatea triunghiului. (0p)

c) Algoritmul descris în curs (cursul 9, slide-urile 18-19) este un algoritm 2-aproximativ pentru forma generală a TSP (pentru instanțele care respectă inegalitatea triunghiului). Verificați dacă în această variantă a problemei, algoritmul din curs este chiar $\frac{3}{2}$ -aproximativ. (1p)

2. Fie P o mulțime de puncte în plan. Din cursurile anterioare știm să construim un *Minimum Spanning Tree* pe baza punctelor din P . Numim acest arbore T . Uneori, adăugând și alte puncte pe lângă cele din P , putem obține un MST cu cost mai mic. Un asemenea arbore, construit prin adăugarea de noduri se numește *Steiner Tree*. Algoritmii pentru calcularea de ST-uri sunt de obicei NP-hard.

a) Arătați că există cazuri în care alegând un punct $q \notin P$ obținem un MST pentru mulțimea de puncte $P \cup \{q\}$ cu un cost mai mic decât T . (1p)

b) Fie Q o mulțime de puncte în plan, disjunctă față de P . Arătați că T este de cel mult două ori mai mare ca și cost față de MST-ul pentru $P \cup Q$. Altfel spus, odată ce avem un MST pentru P , putem îmbunătăți rezultatul adăugând alte puncte, dar niciodată cu mai mult de un factor de 2. (1p)

3. Considerăm problema *k-center metric în plan*: se dă o mulțime P de puncte în plan și un număr întreg k . Trebuie să alegem o mulțime $C \subseteq P$ de cel mult k centre astfel încât să minimizăm valoarea

$$\max_{p \in P} \min_{c \in C} d(p, c),$$

unde $d(p, c)$ este distanța euclidiană dintre punctele p și c .

Cu alte cuvinte, vrem să alegem cel mult k puncte-centru astfel încât orice punct din mulțime să fie cât mai aproape de un centru ales, iar distanța maximă până la cel mai apropiat centru să fie minimă.

De exemplu, dacă

$$P = \{(0, 0), (1, 0), (3, 0), (4, 0)\} \quad \text{și} \quad k = 2,$$

atunci alegerea centrelor

$$C = \{(1, 0), (3, 0)\}$$

are cost 1, deoarece fiecare punct din P se află la distanță cel mult 1 de unul dintre aceste centre.

a) Dați un algoritm de 2-aproximare pentru această problemă. (1p)

b) Arătați că problema nu poate fi aproximată cu niciun factor strict mai mic decât 2, decât dacă $P = NP$. (1p)

Vertex Cover (2p)

Fie $X = \{x_1, x_2, \dots, x_n\}$ o mulțime de variabile boolene. Numim formulă booleană (peste mulțimea X) în *Conjunctive Normal Form* (CNF) o expresie de forma $C_1 \wedge C_2 \wedge \dots \wedge C_m$ unde fiecare predicat (clauză) C_i este o disjuncție a unui număr de variabile (este alcătuit din mai multe

variabile cu simbolul „ $\vee$ ” - *logical or* - între ele).

Exemplu de astfel de expresie:

$$(x_1 \vee x_3 \vee x_4) \wedge (x_2 \vee x_3 \vee x_7) \wedge (x_1 \vee x_5 \vee x_6) \wedge (x_2 \vee x_5 \vee x_7)$$

Evident că orice astfel de expresie va fi evaluată ca `true` dacă toate elementele lui X iau valoarea `true`. Ne interesează în schimb să aflăm numărul minim de elemente din X care trebuie să aibă valoarea `true` astfel încât toată expresia să fie `true`.

Fie următorul algoritm pentru problema de mai sus în varianta în care fiecare clauză are exact trei variabile (numită *3CNF*):

Greedy-3CNF

1. Fie $C = \{ C_1, \dots, C_m \}$ mulțimea de predicate, $X = \{ x_1, \dots, x_n \}$ mulțimea de variabile.
2. Cât timp $C \neq \emptyset$ execută:
 - (a) Alegem aleator $C_j \in C$.
 - (b) Fie x_i una dintre variabilele din C_j .
 - (c) $x_i \leftarrow \text{true}$
 - (d) Eliminăm din C toate predicatele care îl conțin pe x_i
3. Soluția constă din variabilele pe care le-am setat ca `true` pe parcursul execuției algoritmului

Cerințe

- a) Este algoritmul descris mai sus un algoritm aproximativ? În cazul afirmativ, determinați factorul de aproximare (*worst case*) al algoritmului. Altfel, justificați de ce nu este aproximativ. (0,5p)
- b) Modificați algoritmul de mai sus astfel încât acesta să fie un algoritm 3-aproximativ pentru problema inițială (și justificați de ce se obține acest factor în urma modificărilor voastre). (0,5p)
- c) Reformulați problema de mai sus sub forma unei probleme de programare liniară. (0,5p)
- d) Dați o soluție 3-aproximativă care să rezolve problema de programare liniară formulată la subpunctul anterior. (0,5p)